Rails system tests use Selenium by default, so we'll stick with that. I'm opting to go with Chrome via Selenium. Chrome is the most popular desktop browser, and people are saying good things about its headless support compared to Capybara-webkit.[14]

To use Selenium, we'll need to add the selenium-webdriver gem to our Gemfile:

```
group :development, :test do
  # Call 'byebug' anywhere in the code to stop execution and get a debugger…
  gem 'byebug', platforms: [:mri, :mingw, :x64_mingw]
  gem 'rspec-rails', '~> 3.8'
  gem 'capybara', '~> 3.7'
➤ gem 'selenium-webdriver', '~> 3.14'
end
```

and then install it by rebuilding our image and recreating our web container:

```
$ docker-compose build web
$ docker-compose stop web
$ docker-compose up -d --force-recreate web
```

But how are we going to run Chrome with Docker? The same way we typically run any software with Docker—in a container. There are prepared images for running standalone versions of Chrome, and in fact, we're going to use one such image maintained by Selenium itself.[15]

Let's add that to our Compose file:

```
selenium_chrome:
  image: selenium/standalone-chrome-debug
  logging:
    driver: none
  ports:
    - "5900:5900"
```

This adds a new service that we've chosen to call selenium_chrome, whose containers will be based on the selenium/standalone-chrome-debug image. We've opted for the debug,[16] rather than the standard version of this image,[17] because it includes and runs a VNC server. This gives us the option to visually see Chrome running inside the container using a VNC client—useful if you want to actually see the tests running.

We've also turned logging off by setting the logging driver to none because the Selenium Chrome image has noisy output that we don't need to see. We create

14. https://robots.thoughtbot.com/headless-feature-specs-with-chrome
15. https://hub.docker.com/r/selenium/standalone-chrome-debug/
16. https://hub.docker.com/r/selenium/standalone-chrome-debug/
17. https://hub.docker.com/r/selenium/standalone-chrome/